

ESTRATEGIA DE RAMAS

EcoRoute — Reglas para Git (Gitflow)

1. INTRODUCCIÓN

1.1 Propósito

Este documento define las reglas, convenciones y flujos de trabajo para el control de versiones del sistema EcoRoute usando Git. Establece la estrategia de ramas adoptada, los criterios para crear, nombrar, fusionar y eliminar ramas, y los estándares para los mensajes de commit. Su objetivo es garantizar la trazabilidad del desarrollo, la estabilidad del código en producción y la reproducibilidad de los experimentos científicos.

1.2 Estrategia Adoptada

EcoRoute adopta **Gitflow** como estrategia de ramas. Esta decisión responde a las características del proyecto: ciclos de desarrollo por iteraciones bien definidas, necesidad de mantener versiones estables del entorno de experimentos y la importancia de aislar el desarrollo de nuevas funcionalidades del código validado.

1.3 Justificación frente a Trunk-Based Development

Criterio	Gitflow	Trunk-Based
Ciclos de desarrollo	Iteraciones definidas con entregas por versión	Integración continua con despliegues frecuentes
Reproducibilidad científica	Alta: versiones etiquetadas reproducen experimentos exactos	Media: el trunk evoluciona continuamente
Tamaño del equipo	Adecuado para equipos pequeños con roles definidos	Óptimo para equipos grandes con CI/CD maduro
Estabilidad del entorno	Ramas de release aíslan el entorno de experimentos	El trunk puede incluir cambios inestables
Veredicto para EcoRoute	Seleccionado	Descartado

2. ESTRUCTURA DE RAMAS

2.1 Ramas Permanentes

Son ramas que existen durante toda la vida del proyecto y nunca se eliminan.

Rama	Nombre	Propósito
------	--------	-----------

Principal	main	Contiene únicamente versiones estables y etiquetadas del sistema. Todo código en main ha pasado la suite completa de pruebas y está listo para reproducir experimentos. Nadie hace commits directos a main.
Desarrollo	develop	Rama de integración continua del desarrollo. Contiene el trabajo en progreso integrado. Es la base desde la que salen todas las ramas de funcionalidad.

2.2 Ramas Temporales

Son ramas que se crean para un propósito específico y se eliminan tras su fusión.

Tipo	Prefijo	Se crea desde	Se fusiona hacia	Se elimina
Feature	feature/	develop	develop	Tras el merge a develop
Release	release/	develop	main y develop	Tras el merge a main
Hotfix	hotfix/	main	main y develop	Tras el merge a main
Patch	patch/	develop	develop	Tras el merge a develop
Experiment	experiment/	develop	develop (si exitoso)	Tras decisión de integrar o descartar

3. CONVENCIONES DE NOMENCLATURA

3.1 Reglas Generales

- Usar **kebab-case** (minúsculas con guiones): feature/dqn-replay-buffer
- No usar espacios, mayúsculas ni caracteres especiales
- El nombre debe ser descriptivo y corto (máximo 5 palabras)
- Incluir siempre el prefijo del tipo de rama

3.2 Ejemplos por Tipo de Rama

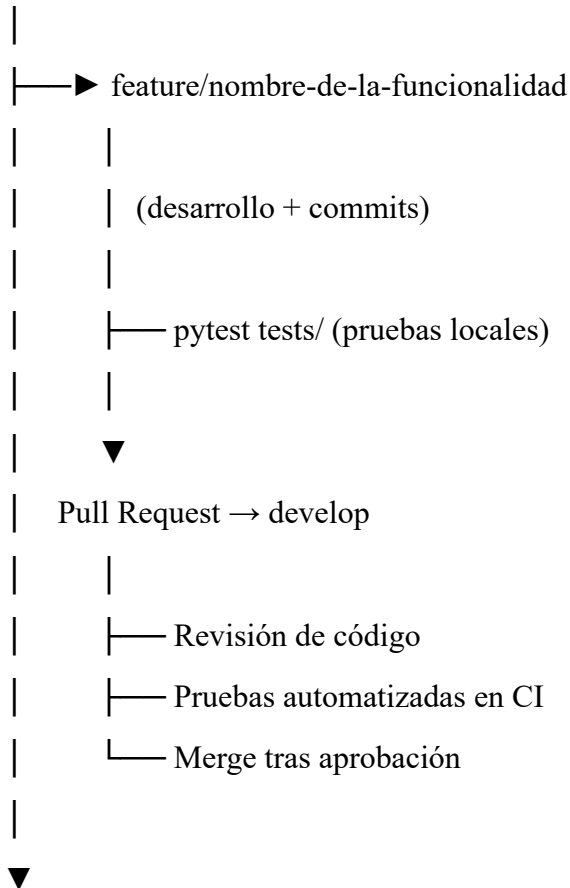
Tipo	Ejemplo correcto	Ejemplo incorrecto
Feature	feature/emission-calculator-vsp	feature/EmissionCalculator
Feature	feature/dqn-target-network	nueva-funcionalidad
Release	release/v1.0.0	release/version-final
Hotfix	hotfix/traci-connection-timeout	fix/bug

Patch	patch/dep-pytorch-2.0.1	patch/actualizacion
Experiment	experiment/double-dqn-comparison	experiment/prueba1

4. FLUJO DE TRABAJO

4.1 Flujo de una Feature

develop



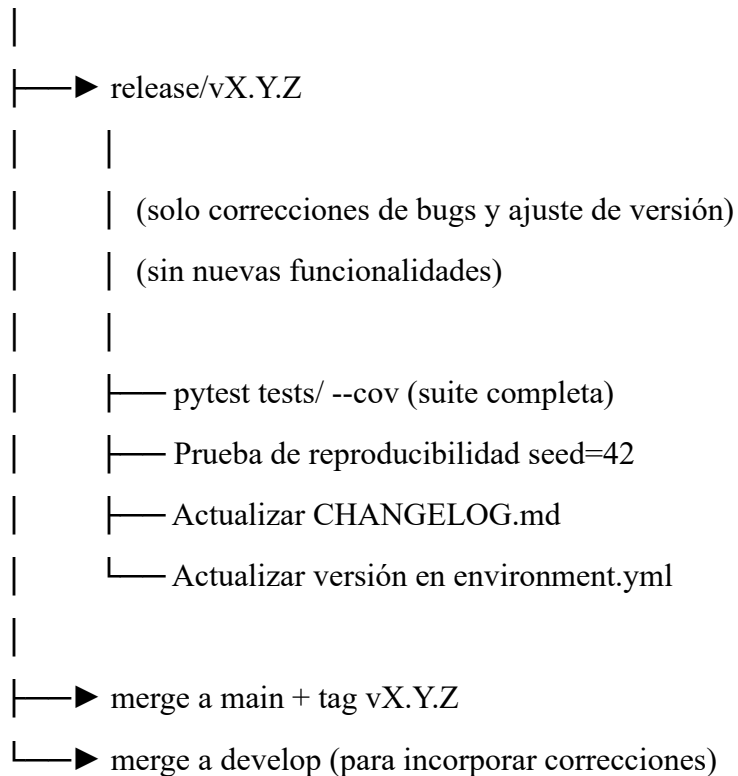
develop (actualizado)

Reglas para ramas feature:

- Una rama feature por funcionalidad o módulo del sistema
- El desarrollador nunca hace merge directo a develop sin Pull Request
- La rama feature debe estar actualizada con develop antes del PR
- Se elimina la rama local y remota tras el merge

4.2 Flujo de una Release

develop (con funcionalidades completas de la iteración)

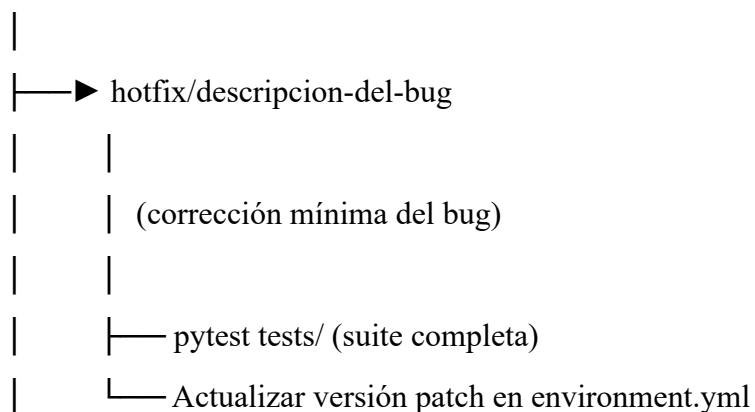


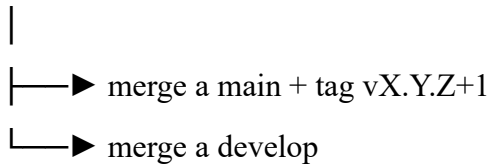
Reglas para ramas release:

- Solo se crean cuando develop tiene todas las funcionalidades de la iteración
- En una rama release no se agregan nuevas funcionalidades
- Solo se permiten correcciones de bugs detectados en la revisión final
- Al hacer merge a main se crea obligatoriamente un tag de versión

4.3 Flujo de un Hotfix

main (versión estable con bug crítico)



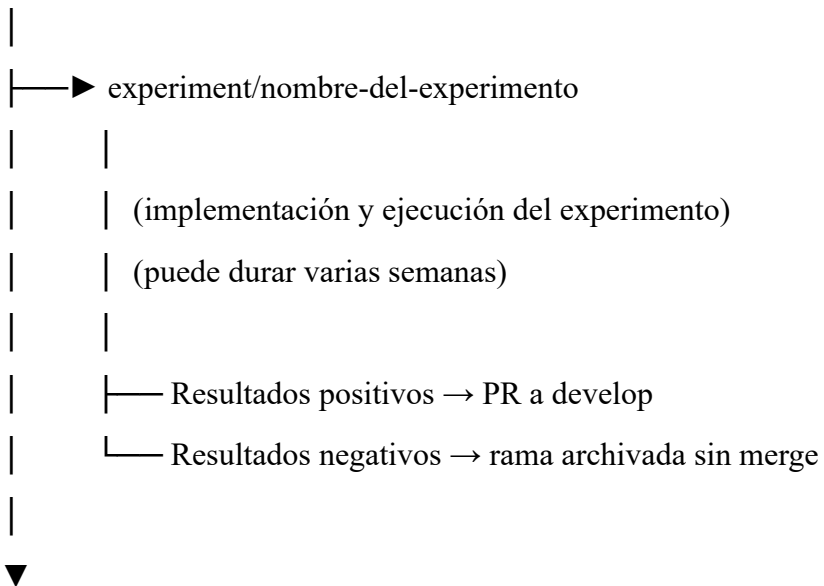


Reglas para ramas hotfix:

- Solo para bugs críticos que afecten la reproducibilidad o la seguridad del sistema
- La corrección debe ser la mínima necesaria para resolver el bug
- Obligatorio hacer merge tanto a main como a develop

4.4 Flujo de un Experimento

develop



develop (si el experimento se integra)

Reglas para ramas experiment:

- Usadas para probar variantes del algoritmo DQN o la función de recompensa
- No tienen plazo obligatorio de merge
- Si el experimento no da resultados útiles la rama se archiva con un tag descriptivo

5. ESTÁNDAR DE COMMITS

5.1 Formato del Mensaje de Commit

<tipo>(<módulo>): <descripción corta en imperativo>

[cuerpo opcional — explica el qué y el por qué, no el cómo]

[footer opcional — referencias a issues o breaking changes]

5.2 Tipos de Commit

Tipo	Cuando usarlo	Ejemplo
feat	Nueva funcionalidad	feat(agent): add target network update logic
fix	Corrección de bug	fix(env): handle TraCI connection timeout
test	Agregar o modificar pruebas	test(emissions): add VSP calculation unit test
refactor	Refactorización sin cambio de comportamiento	refactor(buffer): replace list with deque
docs	Cambios en documentación	docs(config): add hyperparameter comments to yaml
patch	Actualización de dependencia	patch(deps): update pytorch from 2.0.0 to 2.0.1
perf	Mejora de rendimiento	perf(trainer): move plot generation to thread
chore	Tareas de mantenimiento	chore(env): update environment.yml lockfile

5.3 Reglas para Mensajes de Commit

- La descripción corta no supera 72 caracteres
- Usar modo imperativo en inglés: "add", "fix", "update", no "added", "fixed"
- El módulo entre paréntesis corresponde al directorio o archivo principal afectado
- No terminar la descripción con punto
- Commits atómicos: un commit por cambio lógico, no acumular cambios no relacionados

6. ETIQUETADO DE VERSIONES

6.1 Esquema de Versiones (Semantic Versioning)

EcoRoute usa **Semantic Versioning (SemVer)**: vMAJOR.MINOR.PATCH

Componente	Cuándo incrementar	Ejemplo
------------	--------------------	---------

MAJOR	Cambio de algoritmo principal, rediseño de arquitectura o ruptura de reproducibilidad entre versiones	v1.0.0 → v2.0.0
MINOR	Nueva funcionalidad compatible: nuevo módulo, nuevo baseline, nuevo modelo de emisiones	v1.0.0 → v1.1.0
PATCH	Corrección de bug, actualización de dependencia, mejora de documentación	v1.0.0 → v1.0.1

6.2 Tags en Git

Cada versión en main recibe un tag anotado:

bash

```
git tag -a v1.0.0 -m "Release v1.0.0 Entrenamiento DQN intersección aislada"
```

```
git push origin v1.0.0
```

7. REGLAS DE PROTECCIÓN DE RAMAS

Regla	Rama afectada	Descripción
Sin push directo	main, develop	Obligatorio usar Pull Request para cualquier cambio
Aprobación requerida	main	Al menos una revisión de código aprobada antes del merge
Pruebas obligatorias	main, develop	La suite de pytest debe pasar antes de cualquier merge
Tag obligatorio	main	Todo merge a main debe ir acompañado de un tag de versión
Rama actualizada	Todas	La rama debe estar actualizada con su base antes del PR